

16 | EXPECTATION MAXIMIZATION

A hen is only an egg's way of making another egg. – Samuel Butler

SUPPOSE YOU WERE BUILDING a naive Bayes model for a text categorization problem. After you were done, your boss told you that it became prohibitively expensive to obtain labeled data. You now have a probabilistic model that assumes access to labels, but you don't have any labels! Can you still do something?

Amazingly, you can. You can treat the labels as **hidden variables**, and attempt to learn them at the same time as you learn the parameters of your model. A very broad family of algorithms for solving problems just like this is the **expectation maximization** family. In this chapter, you will derive expectation maximization (EM) algorithms for clustering and dimensionality reduction, and then see why EM works.

Learning Objectives:

- Explain the relationship between parameters and hidden variables.
- Construct generative stories for clustering and dimensionality reduction.
- Draw a graph explaining how EM works by constructing convex lower bounds.
- Implement EM for clustering with mixtures of Gaussians, and contrasting it with k -means.
- Evaluate the differences between EM and gradient descent for hidden variable models.

Dependencies:

16.1 Grading an Exam without an Answer Key

Alice's machine learning professor Carlos gives out an exam that consists of 50 true/false questions. Alice's class of 100 students takes the exam and Carlos goes to grade their solutions. If Carlos made an answer key, this would be easy: he would just count the fraction of correctly answered questions each student got, and that would be their score. But, like many professors, Carlos was really busy and didn't have time to make an answer key. Can he still grade the exam?

There are two insights that suggest that he might be able to. Suppose he know ahead of time that Alice was an awesome student, and is basically guaranteed to get 100% on the exam. In that case, Carlos can simply use Alice's answers as the ground truth. More generally, if Carlos assumes that *on average* students are better than random guessing, he can hope that the majority answer for each question is likely to be correct. Combining this with the previous insight, when doing the "voting", he might want to pay more attention to the answers of the better students.

To be a bit more pedantic, suppose there are $N = 100$ students and $M = 50$ questions. Each student n has a score s_n , between 0 and

1 that denotes how well they do on the exam. The score is what we really want to compute. For each question m and each student n , the student has provided an answer $a_{n,m}$, which is either zero or one. There is also an unknown ground truth answer for each question m , which we'll call t_m , which is also either zero or one.

As a starting point, let's consider a simple heuristic and then complexify it. The heuristic is the "majority vote" heuristic and works as follows. First, we estimate t_m as the most common answer for question m : $t_m = \operatorname{argmax}_t \sum_n \mathbf{1}[a_{n,m} = t]$. Once we have a guess for each true answer, we estimate each students' score as how many answers they produced that match this guessed key: $s_n = \frac{1}{M} \sum_m \mathbf{1}[a_{n,m} = t_m]$.

Once we have these scores, however, we might want to trust some of the students more than others. In particular, answers from students with high scores are perhaps more likely to be correct, so we can *recompute* the ground truth, according to weighted votes. The weight of the votes will be precisely the score the corresponding each student:

$$t_m = \operatorname{argmax}_t \sum_n s_n \mathbf{1}[a_{n,m} = t] \quad (16.1)$$

You can recognize this as a *chicken and egg problem*. If you knew the student's scores, you could estimate an answer key. If you had an answer key, you could compute student scores. A very common strategy in computer science for dealing with such chicken and egg problems is to iterate. Take a guess at the first, compute the second, recompute the first, and so on.

In order to develop this idea formally, we have to case the problem in terms of a probabilistic model with a generative story. The generative story we'll use is:

1. For each question m , choose a true answer $t_m \sim \operatorname{Ber}(0.5)$
2. For each student n , choose a score $s_n \sim \operatorname{Uni}(0, 1)$
3. For each question m and each student n , choose an answer $a_{n,m} \sim \operatorname{Ber}(s_n)^{t_m} \operatorname{Ber}(1 - s_n)^{1-t_m}$

In the first step, we generate the true answers independently by flipping a fair coin. In the second step, each students' overall score is determined to be a uniform random number between zero and one. The tricky step is step three, where each students' answer is generated for each question. Consider student n answering question m , and suppose that $s_n = 0.9$. If $t_m = 1$, then $a_{n,m}$ should be 1 (i.e., correct) 90% of the time; this can be accomplished by drawing the answer from $\operatorname{Ber}(0.9)$. On the other hand, if $t_m = 0$, then $a_{n,m}$ should be 0 (i.e., incorrect) 90% of the time; this can be accomplished by drawing

the answer from $\mathcal{Ber}(0.1)$. The exponent in step 3 selects which of two Bernoulli distributions to draw from, and then implements this rule.

This can be translated into the following likelihood:

$$\begin{aligned}
 p(\mathbf{a}, \mathbf{t}, \mathbf{s}) &= \left[\prod_m 0.5^{t_m} 0.5^{1-t_m} \right] \times \left[\prod_n 1 \right] \\
 &\times \left[\prod_n \prod_m s_n^{a_{n,m} t_m} (1-s_n)^{(1-a_{n,m}) t_m} \right. \\
 &\quad \left. s_n^{(1-a_{n,m})(1-t_m)} (1-s_n)^{a_{n,m}(1-t_m)} \right] \tag{16.2}
 \end{aligned}$$

$$= 0.5^M \prod_n \prod_m s_n^{a_{n,m} t_m} (1-s_n)^{(1-a_{n,m}) t_m} s_n^{(1-a_{n,m})(1-t_m)} (1-s_n)^{a_{n,m}(1-t_m)} \tag{16.3}$$

Suppose we knew the true labels $\mathbf{t}$. We can take the log of this likelihood and differentiate it with respect to the score s_n of some student (note: we can drop the 0.5^M term because it is just a constant):

$$\begin{aligned}
 \log p(\mathbf{a}, \mathbf{t}, \mathbf{s}) &= \sum_n \sum_m \left[a_{n,m} t_m \log s_n + (1-a_{n,m})(1-t_m) \log(s_n) \right. \\
 &\quad \left. + (1-a_{n,m}) t_m \log(1-s_n) + a_{n,m}(1-t_m) \log(1-s_n) \right] \tag{16.4}
 \end{aligned}$$

$$\frac{\partial \log p(\mathbf{a}, \mathbf{t}, \mathbf{s})}{\partial s_n} = \sum_m \left[\frac{a_{n,m} t_m + (1-a_{n,m})(1-t_m)}{s_n} - \frac{(1-a_{n,m}) t_m + a_{n,m}(1-t_m)}{1-s_n} \right] \tag{16.5}$$

The derivative has the form $\frac{A}{s_n} - \frac{B}{1-s_n}$. If we set this equal to zero and solve for s_n , we get an optimum of $s_n = \frac{A}{A+B}$. In this case:

$$A = \sum_m [a_{n,m} t_m + (1-a_{n,m})(1-t_m)] \tag{16.6}$$

$$B = \sum_m [(1-a_{n,m}) t_m + a_{n,m}(1-t_m)] \tag{16.7}$$

$$A + B = \sum_m [1] = M \tag{16.8}$$

Putting this together, we get:

$$s_n = \frac{1}{M} \sum_m [a_{n,m} t_m + (1-a_{n,m})(1-t_m)] \tag{16.9}$$

In the case of known $\mathbf{t}$ s, this matches exactly what we had in the heuristic.

However, we do not know $\mathbf{t}$, so instead of using the “true” values of $\mathbf{t}$, we’re going to use their *expectations*. In particular, we will compute s_n by maximizing its likelihood under the *expected* values

of t , hence the name **expectation maximization**. If we are going to compute expectations of t , we have to say: expectations according to which probability distribution? We will use the distribution $p(t_m | \mathbf{a}, \mathbf{s})$. Let $\tilde{t}_m$ denote $\mathbb{E}_{t_m \sim p(t_m | \mathbf{a}, \mathbf{s})}[t_m]$. Because t_m is a binary variable, its expectation is equal to its probability; namely: $\tilde{t}_m = p(t_m | \mathbf{a}, \mathbf{s})$.

How can we compute this? We will compute $C = p(t_m = 1, \mathbf{a}, \mathbf{s})$ and $D = p(t_m = 0, \mathbf{a}, \mathbf{s})$ and then compute $\tilde{t}_m = C/(C + D)$. The computation is straightforward:

$$C = 0.5 \prod_n s_n^{a_{n,m}} (1 - s_n)^{1-a_{n,m}} = 0.5 \prod_{\substack{n: \\ a_{n,m}=1}} s_n \prod_{\substack{n: \\ a_{n,m}=0}} (1 - s_n) \quad (16.10)$$

$$D = 0.5 \prod_n s_n^{1-a_{n,m}} (1 - s_n)^{a_{n,m}} = 0.5 \prod_{\substack{n: \\ a_{n,m}=1}} (1 - s_n) \prod_{\substack{n: \\ a_{n,m}=0}} s_n \quad (16.11)$$

If you inspect the value of C , it is basically “voting” (in a product form, not a sum form) the scores of those students who agree that the answer is 1 with one-minus-the-score of those students who do not. The value of D is doing the reverse. This is a form of multiplicative voting, which has the effect that if a given student has a perfect score of 1.0, their results will carry the vote completely.

We now have a way to:

1. Compute expected ground truth values $\tilde{t}_m$, given scores.
2. Optimize scores s_n given expected ground truth values.

The full solution is then to alternate between these two. You can start by initializing the ground truth values at the majority vote (this seems like a safe initialization). Given those, compute new scores. Given those new scores, compute new ground truth values. And repeat until tired.

In the next two sections, we will consider a more complex unsupervised learning model for clustering, and then a generic mathematical framework for expectation maximization, which will answer questions like: will this process converge, and, if so, to what?

16.2 Clustering with a Mixture of Gaussians

In Chapter 9, you learned about probabilistic models for classification based on density estimation. Let’s start with a fairly simple classification model that *assumes* we have labeled data. We will shortly remove this assumption. Our model will state that we have K classes, and data from class k is drawn from a Gaussian with mean μ_k and variance σ_k^2 . The choice of classes is parameterized by θ . The generative story for this model is:

1. For each example $n = 1 \dots N$:

(a) Choose a label $y_n \sim \text{Disc}(\theta)$

(b) Choose example $x_n \sim \text{Nor}(\mu_{y_n}, \sigma_{y_n}^2)$

This generative story can be directly translated into a likelihood as before:

$$p(D) = \prod_n \text{Mult}(y_n \mid \theta) \text{Nor}(x_n \mid \mu_{y_n}, \sigma_{y_n}^2) \quad (16.12)$$

$$= \prod_n \underbrace{\theta_{y_n}}_{\text{choose label}} \underbrace{\left[2\pi\sigma_{y_n}^2 \right]^{-\frac{D}{2}} \exp \left[-\frac{1}{2\sigma_{y_n}^2} \|x_n - \mu_{y_n}\|^2 \right]}_{\text{choose feature values}} \quad (16.13)$$

If you had access to labels, this would be all well and good, and you could obtain closed form solutions for the maximum likelihood estimates of all parameters by taking a log and then taking gradients of the log likelihood:

$$\theta_k = \text{fraction of training examples in class } k \quad (16.14)$$

$$= \frac{1}{N} \sum_n [y_n = k]$$

$$\mu_k = \text{mean of training examples in class } k \quad (16.15)$$

$$= \frac{\sum_n [y_n = k] x_n}{\sum_n [y_n = k]}$$

$$\sigma_k^2 = \text{variance of training examples in class } k \quad (16.16)$$

$$= \frac{\sum_n [y_n = k] \|x_n - \mu_k\|^2}{\sum_n [y_n = k]}$$

Suppose that you *don't* have labels. Analogously to the K -means algorithm, one potential solution is to iterate. You can start off with guesses for the values of the unknown variables, and then iteratively improve them over time. In K -means, the approach was the *assign* examples to labels (or clusters). This time, instead of making hard assignments (“example 10 belongs to cluster 4”), we’ll make **soft assignments** (“example 10 belongs half to cluster 4, a quarter to cluster 2 and a quarter to cluster 5”). So as not to confuse ourselves too much, we’ll introduce a new variable, $z_n = \langle z_{n,1}, \dots, z_{n,K} \rangle$ (that sums to one), to denote a fractional assignment of examples to clusters.

This notion of soft-assignments is visualized in Figure 16.1. Here, we’ve depicted each example as a pie chart, and its coloring denotes the degree to which it’s been assigned to each (of three) clusters. The size of the pie pieces correspond to the z_n values.

? You should be able to derive the maximum likelihood solution results formally by now.

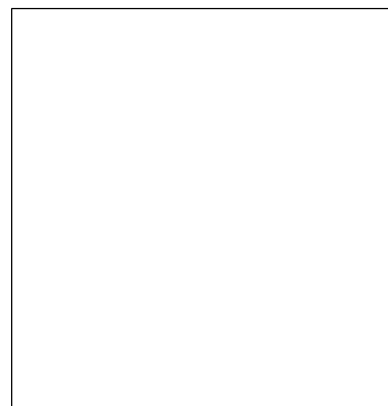


Figure 16.1: em:piecharts: A figure showing pie charts

Formally, $z_{n,k}$ denotes the probability that example n is assigned to cluster k :

$$z_{n,k} = p(y_n = k \mid \mathbf{x}_n) \quad (16.17)$$

$$= \frac{p(y_n = k, \mathbf{x}_n)}{p(\mathbf{x}_n)} \quad (16.18)$$

$$= \frac{1}{Z_n} \text{Mult}(k \mid \boldsymbol{\theta}) \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \sigma_k^2) \quad (16.19)$$

Here, the normalizer Z_n is to ensure that z_n sums to one.

Given a set of parameters (the θ s, μ s and σ^2 s), the **fractional assignments** $z_{n,k}$ are easy to compute. Now, akin to K -means, given fractional assignments, you need to recompute estimates of the model parameters. In analogy to the maximum likelihood solution (Eqs (??)-(??)), you can do this by counting fractional points rather than full points. This gives the following re-estimation updates:

$$\theta_k = \text{fraction of training examples in class } k \quad (16.20)$$

$$= \frac{1}{N} \sum_n z_{n,k}$$

$$\boldsymbol{\mu}_k = \text{mean of fractional examples in class } k \quad (16.21)$$

$$= \frac{\sum_n z_{n,k} \mathbf{x}_n}{\sum_n z_{n,k}}$$

$$\sigma_k^2 = \text{variance of fractional examples in class } k \quad (16.22)$$

$$= \frac{\sum_n z_{n,k} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2}{\sum_n z_{n,k}}$$

All that has happened here is that the hard assignments “[$y_n = k$]” have been replaced with soft assignments “ $z_{n,k}$ ”. As a bit of foreshadowing of what is to come, what we’ve done is essentially replace known labels with *expected labels*, hence the name “expectation maximization.”

Putting this together yields Algorithm 16.2. This is the **GMM** (“**Gaussian Mixture Models**”) algorithm, because the probabilistic model being learned describes a dataset as being drawn from a mixture distribution, where each component of this distribution is a Gaussian.

Just as in the K -means algorithm, this approach is susceptible to local optima and quality of initialization. The heuristics for computing better initializers for K -means are also useful here.

?

Aside from the fact that GMMs use soft assignments and K -means uses hard assignments, there are other differences between the two approaches. What are they?

16.3 The Expectation Maximization Framework

At this point, you’ve seen a method for learning in a particular probabilistic model with hidden variables. Two questions remain: (1) can

Algorithm 38 GMM($\mathbf{X}, K$)

```

1: for  $k = 1$  to  $K$  do
2:    $\mu_k \leftarrow$  some random location      // randomly initialize mean for  $k$ th cluster
3:    $\sigma_k^2 \leftarrow 1$                   // initialize variances
4:    $\theta_k \leftarrow 1/K$                 // each cluster equally likely a priori
5: end for
6: repeat
7:   for  $n = 1$  to  $N$  do
8:     for  $k = 1$  to  $K$  do
9:        $z_{n,k} \leftarrow \theta_k [2\pi\sigma_k^2]^{-\frac{D}{2}} \exp\left[-\frac{1}{2\sigma_k^2} \|\mathbf{x}_n - \mu_k\|^2\right]$  // compute
        (unnormalized) fractional assignments
10:    end for
11:     $\mathbf{z}_n \leftarrow \frac{1}{\sum_k z_{n,k}} \mathbf{z}_n$  // normalize fractional assignments
12:  end for
13:  for  $k = 1$  to  $K$  do
14:     $\theta_k \leftarrow \frac{1}{N} \sum_n z_{n,k}$  // re-estimate prior probability of cluster  $k$ 
15:     $\mu_k \leftarrow \frac{\sum_n z_{n,k} \mathbf{x}_n}{\sum_n z_{n,k}}$  // re-estimate mean of cluster  $k$ 
16:     $\sigma_k^2 \leftarrow \frac{\sum_n z_{n,k} \|\mathbf{x}_n - \mu_k\|^2}{\sum_n z_{n,k}}$  // re-estimate variance of cluster  $k$ 
17:  end for
18: until converged
19: return  $\mathbf{z}$  // return cluster assignments

```

you apply this idea more generally and (2) why is it even a reasonable thing to do? Expectation maximization is a *family* of algorithms for performing maximum likelihood estimation in probabilistic models with hidden variables.

The general flavor of how we will proceed is as follows. We want to maximize the log likelihood $\mathcal{L}$, but this will turn out to be difficult to do directly. Instead, we'll pick a surrogate function $\tilde{\mathcal{L}}$ that's a lower bound on $\mathcal{L}$ (i.e., $\tilde{\mathcal{L}} \leq \mathcal{L}$ everywhere) that's (hopefully) easier to maximize. We'll construct the surrogate in such a way that increasing it will force the true likelihood to also go up. After maximizing $\tilde{\mathcal{L}}$, we'll construct a *new* lower bound and optimize that. This process is shown pictorially in Figure 16.2.

To proceed, consider an arbitrary probabilistic model $p(\mathbf{x}, \mathbf{y} \mid \boldsymbol{\theta})$, where $\mathbf{x}$ denotes the observed data, $\mathbf{y}$ denotes the hidden data and $\boldsymbol{\theta}$ denotes the parameters. In the case of Gaussian Mixture Models, $\mathbf{x}$ was the data points, $\mathbf{y}$ was the (unknown) labels and $\boldsymbol{\theta}$ included the cluster prior probabilities, the cluster means and the cluster variances. Now, given access *only* to a number of examples $\mathbf{x}_1, \dots, \mathbf{x}_N$, you would like to estimate the parameters ($\boldsymbol{\theta}$) of the model.

Probabilistically, this means that some of the variables are unknown and therefore you need to marginalize (or sum) over their possible values. Now, your data consists only of $\mathbf{X} = \langle \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N \rangle$,

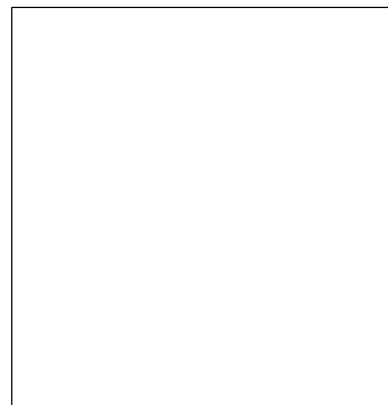


Figure 16.2: em:lowerbound: A figure showing successive lower bounds

not the (x, y) pairs in D . You can then write the likelihood as:

$$p(\mathbf{X} \mid \theta) = \sum_{y_1} \sum_{y_2} \cdots \sum_{y_N} p(\mathbf{X}, y_1, y_2, \dots, y_N \mid \theta) \quad \text{marginalization} \quad (16.23)$$

$$= \sum_{y_1} \sum_{y_2} \cdots \sum_{y_N} \prod_n p(x_n, y_n \mid \theta) \quad \text{examples are independent} \quad (16.24)$$

$$= \prod_n \sum_{y_n} p(x_n, y_n \mid \theta) \quad \text{algebra} \quad (16.25)$$

At this point, the natural thing to do is to take logs and then start taking gradients. However, once you start taking logs, you run into a problem: the log cannot eat the sum!

$$\mathcal{L}(\mathbf{X} \mid \theta) = \sum_n \log \sum_{y_n} p(x_n, y_n \mid \theta) \quad (16.26)$$

Namely, the log gets “stuck” outside the sum and cannot move in to decompose the rest of the likelihood term!

The next step is to apply the somewhat strange, but strangely useful, trick of multiplying by 1. In particular, let $q(\cdot)$ be an arbitrary probability distribution. We will multiply the $p(\dots)$ term above by $q(y_n)/q(y_n)$, a valid step so long as q is never zero. This leads to:

$$\mathcal{L}(\mathbf{X} \mid \theta) = \sum_n \log \sum_{y_n} q(y_n) \frac{p(x_n, y_n \mid \theta)}{q(y_n)} \quad (16.27)$$

We will now construct a lower bound using **Jensen’s inequality**.

This is a very useful (and easy to prove!) result that states that $f(\sum_i \lambda_i x_i) \geq \sum_i \lambda_i f(x_i)$, so long as (a) $\lambda_i \geq 0$ for all i , (b) $\sum_i \lambda_i = 1$, and (c) f is concave. If this looks familiar, that’s just because it’s a direct result of the definition of **concavity**. Recall that f is concave if $f(ax + by) \geq af(x) + bf(y)$ whenever $a + b = 1$.

You can now apply Jensen’s inequality to the log likelihood by identifying the list of $q(y_n)$ s as the λ s, log as f (which is, indeed, concave) and each “ x ” as the p/q term. This yields:

$$\mathcal{L}(\mathbf{X} \mid \theta) \geq \sum_n \sum_{y_n} q(y_n) \log \frac{p(x_n, y_n \mid \theta)}{q(y_n)} \quad (16.28)$$

$$= \sum_n \sum_{y_n} \left[q(y_n) \log p(x_n, y_n \mid \theta) - q(y_n) \log q(y_n) \right] \quad (16.29)$$

$$\triangleq \tilde{\mathcal{L}}(\mathbf{X} \mid \theta) \quad (16.30)$$

Note that this inequality holds for *any* choice of function q , so long as its non-negative and sums to one. In particular, it needn’t even be the

? Prove Jensen’s inequality using the definition of concavity and induction.

same function q for each n . We will need to take advantage of both of these properties.

We have succeeded in our first goal: constructing a lower bound on $\mathcal{L}$. When you go to optimize this lower bound for θ , the only part that matters is the first term. The second term, $q \log q$, drops out as a function of θ . This means that the the maximization you need to be able to compute, for fixed q_n s, is:

$$\theta^{(\text{new})} \leftarrow \arg \max_{\theta} \sum_n \sum_{y_n} q_n(y_n) \log p(x_n, y_n \mid \theta) \quad (16.31)$$

This is *exactly* the sort of maximization done for Gaussian mixture models when we recomputed new means, variances and cluster prior probabilities.

The second question is: what should $q_n(\cdot)$ actually be? Any reasonable q will lead to a lower bound, so in order to choose one q over another, we need another criterion. Recall that we are hoping to maximize $\mathcal{L}$ by instead maximizing a lower bound. In order to ensure that an increase in the lower bound implies an increase in $\mathcal{L}$, we need to ensure that $\mathcal{L}(\mathbf{X} \mid \theta) = \tilde{\mathcal{L}}(\mathbf{X} \mid \theta)$. In words: $\tilde{\mathcal{L}}$ should be a lower bound on $\mathcal{L}$ that makes contact at the current point, θ .

16.4 Further Reading

TODO further reading